

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт цифрового развития
Кафедра инфокоммуникаций

ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ РАБОТЕ №1
дисциплины «Программирование на Python»

Выполнил:
Безруков Даниил Андреевич
2 курс, группа ИТС-б-з-20-1,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Руководитель практики:
Воронкин Р.А., канд. физ.-мат. наук,
доцент, доцент кафедры
инфокоммуникаций

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2025 г.

Тема: исследование основных возможностей Git и GitHub

Цель: исследовать базовые возможности системы контроля версий Git и веб-сервиса для хостинга GitHub

Порядок выполнения работы:

Ссылка на репозиторий: <https://github.com/pokachtononame/intro-to-github>

1. Создаём публичный репозиторий согласно заданию 1

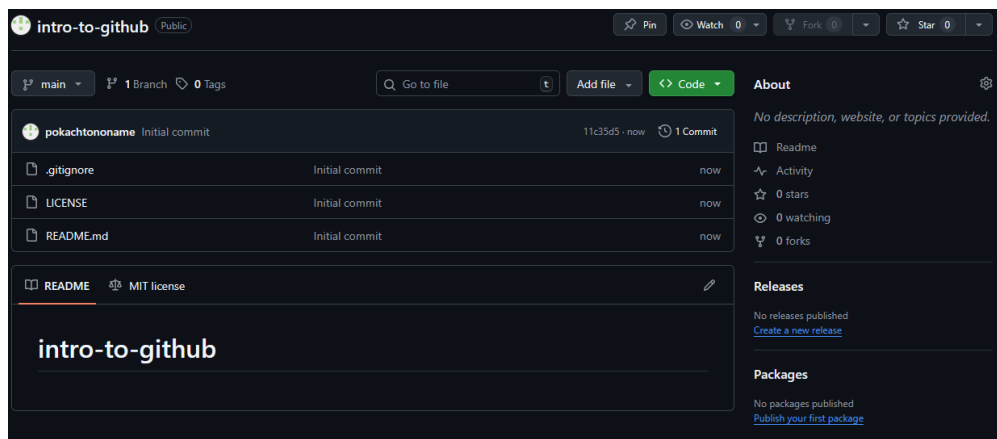


Рисунок 1. Созданный репозиторий

2. Клонировем репозиторий на компьютер

```
MINGW64/c/Users/user
See 'git help git' for an overview of the system.

user@DESKTOP-E6J708F MINGW64 ~
$ git clone https://github.com/pokachtononame/intro-to-github.git
git: 'clone' is not a git command. See 'git --help'.

The most similar command is
  clone

user@DESKTOP-E6J708F MINGW64 ~
$ git clone https://github.com/pokachtononame/intro-to-github.git
Cloning into 'intro-to-github'...
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (5/5), done.

user@DESKTOP-E6J708F MINGW64 ~
$
```

Рисунок 2. Клонирование репозитория

3. Добавляем информацию о группе с помощью команд commit/push

```
user@DESKTOP-E6J708F MINGW64 ~/intro-to-github (main)
$ git commit -m "Add information about local repository"
[main 553f725] Add information about local repository
1 file changed, 2 insertions(+), 1 deletion(-)

user@DESKTOP-E6J708F MINGW64 ~/intro-to-github (main)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)

nothing to commit, working tree clean

user@DESKTOP-E6J708F MINGW64 ~/intro-to-github (main)
$ git push
info: please complete authentication in your browser...
```

Рисунок 3. Добавляем файл README

4. Добавляем в файл .gitignore служебные файлы среды разработки

```
*.vcxproj
*.filters
*.user
*.sln
**/FileContentIndex/
**/v17/
**/.vs/
**/x64/
```

Рисунок 4. Игнорируемые директории и файлы

5. Создаём проект и пишем код, сделав не менее 7 коммитов

```
user@DESKTOP-E6J708F MINGW64 ~/intro-to-github (main)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 7 commits.
(use "git push" to publish your local commits)

nothing to commit, working tree clean
```

Рисунок 5. Локальный репозиторий опережает удалённый

6. Отправляем изменения в удалённый репозиторий

```
user@DESKTOP-E6J708F MINGW64 ~/intro-to-github (main)
$ git push
Enumerating objects: 34, done.
Counting objects: 100% (34/34), done.
Delta compression using up to 8 threads
Compressing objects: 100% (25/25), done.
Writing objects: 100% (32/32), 3.85 KiB | 562.00 KiB/s, done.
Total 32 (delta 12), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (12/12), completed with 1 local object.
To https://github.com/pokachtononame/intro-to-github.git
553f725..6b1fa72 main -> main
```

Рисунок 6. Синхронизация репозитория

7. Проверяем изменения на GitHub

```
7      void input()
11      {
12          std::cout << " | "; in = _getch(); a2 = (int)(in - 48); std::cout << in << " "; in = _getch(); b2 = (int)(in - 48); std::cout << in << " |\n";
13      }
14      void output()
15      {
16          std::cout << " | " << a1 << " ", " << b1 << " |\n";
17          std::cout << " | " << a2 << " ", " << b2 << " |\n";
18      }
19      Matrix2l operator * (const Matrix2l& op)
20      {
21          return Matrix2l{
22              a1 * op.a1 + b1 * op.a2,
23              a1 * op.b1 + b1 * op.b2,
24              a2 * op.a1 + b2 * op.a2,
25              a2 * op.b1 + b2 * op.b2
26          };
27      }
28      Matrix2l operator + (const Matrix2l& op)
29      {
30          return Matrix2l{
31              a1 + op.a1, b1 + op.b1,
32              a2 + op.a2, b2 + op.b2
33          };
34      }
35      };
36
37      struct Complexi
38      {
39          int r;
40          int i;
41          void input()
42          {
43              std::cin >> r; std::cout << "i = "; std::cin >> i;
44          }
45          void output()
46          {
47              const char* sign = (i < 0) ? "-" : "+";
48              std::cout << r << " " << *sign << " " << abs(i) << "i\n";
49          }
50          Complexi operator * (const Complexi& op)
51          {
52              return Complexi{
```

Рисунок 7. Изменения зафиксированы

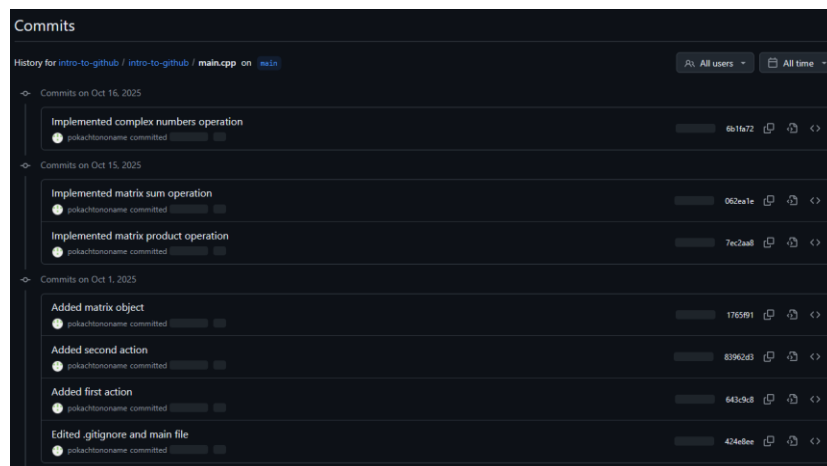


Рисунок 8. История коммитов

Ответы на контрольные вопросы:

1. Что такое СКВ и каково её назначение?

Система управления версиями позволяет хранить несколько версий одного и того же файла, при необходимости возвращаться к более ранним версиям, определять, кто и когда сделал то или иное изменение.

2. В чём недостатки локальных и централизованных СКВ?

Недостатки локальных систем контроля версий заключаются в отсутствии инструментов для командной работы и в меньшей надёжности хранения файлов, так как все версии хранятся локально и могут быть утрачены в случае неисправности.

3. К какой СКВ относится Git?

Git относится к распределённой системе контроля версий. Большинство других СКВ используют централизованную систему.

4. В чём концептуальное отличие Git от других СКВ?

Различия заключаются в следующих моментах:

- Распределённая архитектура. Каждый пользователь хранит у себя полную копию всего репозитория, включающую всю историю изменений, ветки и метаданные. Это позволяет выполнять работу полностью локально.
- Хранение данных. Git хранит версии в виде полных снимков файловой системы, а не историю изменений, как в других СКВ.
- Ветвление. Git предоставляет возможности создавать и объединять ветки, что облегчает работу в команде. В других СКВ возможности ветвления реализованы более сложно и менее гибко.

5. Как обеспечивается целостность хранимых данных в Git?

Каждый объект в Git идентифицируется и проверяется через хеш функцию SHA-1. При чтении любого объекта Git сравнивает сохранённый хеш с перерасчитанным по содержимому.

6. В каких состояниях могут находиться файлы в Git? Как связаны эти состояния?

Основные состояния файлов в Git:

Untracked (неотслеживаемый) — Файл есть в рабочем каталоге, но еще не добавлен в индекс (staging area). Git не следит за этим файлом, он не включен в коммиты.

Unstaged (неподготовленный к коммиту) — Файл уже добавлен в индекс (git add), но изменения еще не зафиксированы.

Staged (подготовленный для коммита) — Файл внесен в индекс (staging area). В этом состоянии файл готов к фиксации с помощью git commit.

Committed (зафиксированный) — Изменения внутри файла зафиксированы и сохранены в истории репозитория.

Modified (отличается от последнего коммита) — Файл был изменен после последнего коммита, но эти изменения еще не добавлены в индекс.

7. Что такое профиль пользователя в GitHub?

Профиль пользователя в GitHub — это индивидуальная страница, которая отображает информацию о конкретном пользователе этой платформы. Он содержит основные сведения, такие как: имя и никнейм, фото профиля, биографию и контактные данные (если указаны), ссылки на внешние ресурсы, например, личные сайты или социальные сети, репозитории пользователя — публичные проекты, созданные или поддерживаемые им, статистику активностей, например, вкладки, количество коммитов, открытых и закрытых Issues, Pull Requests, строку "Публичное" и

"Общее" (Followers и Following) — количество подписчиков и тех, на кого подписан пользователь, вклад в открытый код.

8. Какие бывают репозитории в GitHub?

Публичные репозитории: доступны для просмотра и клонирования всем пользователям Интернета.

Приватные репозитории: доступны только определённым пользователям или командам, которым предоставлены права.

Шаблонные репозитории: предназначены для быстрого создания новых репозиториях на основе заданного шаблона.

Форк-репозитории: это копии чужих репозиториях, создаваемые для внесения изменений и дальнейшего предложения их обратно владельцу (Pull Requests).

Архивные репозитории: это репозитории, которые больше не развиваются и поставлены в состояние "только для чтения".

9. Укажите основные этапы модели работы с GitHub

- 1) Создание репозитория
- 2) Клонирование репозитория на локальный компьютер
- 3) Разработка и внесение изменений
- 4) Фиксация изменений (Commit)
- 5) Обновление удалённого репозитория (Push)
- 6) Создание веток (по желанию)
- 7) Обсуждение и совместная работа
- 8) Объединение изменений (Merge)
- 9) Обновление локальной копии (Pull)
- 10) Решение конфликтов и финализация

10. Как осуществляется первоначальная настройка Git после установки?

После установки Git первоначальная настройка заключается в настройке имени пользователя и электронной почты, что необходимо для правильной идентификации коммитов.

Осуществляется настройка следующими командами:

```
git config --global user.name "Ваше Имя"
```

```
git config --global user.email "ваша@почта.com"
```

11. Опишите этапы создания репозитория в GitHub

- 1) Вход в аккаунт GitHub
- 2) Нажатие на кнопку New Repository
- 3) Настройка параметров репозитория и создание репозитория
- 4) Клонирование репозитория на локальный компьютер

12. Какие типы лицензий поддерживаются в GitHub при создании репозитория?

- 1) MIT License
- 2) Apache License 2.0
- 3) GNU General Public License v3.0 (GPL-3.0)
- 4) GNU Lesser General Public License v3.0 (LGPL-3.0)
- 5) BSD 2-Clause "Simplified" License
- 6) Creative Commons Licenses

13. Как осуществляется клонирование репозитория GitHub? Зачем нужно клонировать репозиторий?

Клонирование осуществляется командой:

```
git clone https://github.com/your_username/repository_name.git
```


Работа с локальным репозиторием позволяет добавлять файлы, делать коммиты и просматривать историю изменений оффлайн, что позволяет допускать меньше ошибок, попадающих в основной репозиторий

14. Как проверить состояние локального репозитория Git?

Состояние локального репозитория можно проверить командой `git status`. Эта команда отображает текущее состояние репозитория, изменения в файлах (не только подготовленные к коммиту, но и незапланированные). Команды `git log` и `git diff` показывают историю коммитов и разницу между текущими изменениями и предыдущей версией соответственно.

15. Как изменяется состояние локального репозитория Git после выполнения следующих операций: добавления/изменения файла в локальный репозиторий Git; добавления нового измененного файла под версионный контроль с помощью команды `git add`; фиксации (коммита) изменений с помощью команды `git commit` и отправки изменений на сервер с помощью команды `git push`?

На первом этапе добавленные и изменённые файлы отображаются в статусе как `untracked` (неотслеживаемый) и `modified` (изменённый).

На втором этапе изменения попадают в индекс (`staging area`). Команда `git status` показывает такие файлы как `staged` (готовые к коммиту).

На третьем этапе файлы, отмеченные как `staged` попадают в коммит, т. е. изменения фиксируются, а история изменений обновляется. После коммита индекс очищается, и в статусе отображается, что изменений нет.

На четвёртом этапе коммит из локального репозитория отправляется на удалённый сервер и состояние репозитория синхронизируется. В статусе отображается, что локальная ветка совпадает с удалённой.

16. У Вас имеется репозиторий на GitHub и два рабочих компьютера, с помощью которых вы можете осуществлять работу над некоторым проектом с использованием этого репозитория. Опишите последовательность команд, с помощью которых оба локальных репозитория, связанных с репозиторием GitHub будут находиться в синхронизированном состоянии.

Сначала на обоих компьютерах клонируем удалённый репозиторий командой `git clone`. После клонирования на двух компьютерах можно работать над проектом и делать коммиты. Перед началом работы на втором компьютере нужно обновить локальный репозиторий командой `git pull`.

Итоговая схема для обеих машин:

- 1) Перед тем, как начать работу — выполнить `git pull`, чтобы загрузить все последние изменения.
- 2) После внесения изменений — выполнить `git add`, `git commit`.
- 3) Отправить свои изменения командой `git push`.

17. GitHub является не единственным сервисом, работающим с Git. Какие ещё сервисы Вам известны? Приведите сравнительный анализ одного из таких сервисов с GitHub.

Сравнительный анализ GitHub и GitLab

Общая концепция:

GitHub ориентирован на open-source проекты, имеет большое число открытых репозиторий, инструменты для взаимодействия с сообществом.

GitLab имеет встроенные CI/CD инструменты, а также позволяет размещение на собственном сервере, что делает его более предпочтительным для больших команд.

Лицензирование и открытость:

Исходный код GitHub закрыт. Открытые репозитории бесплатны, приватные репозитории ограниченно бесплатны.

Исходный код GitLab полностью открыт, что позволяет существовать self-hosted решениям. Приватные репозитории в GitLab бесплатны.

18. Интерфейс командной строки является не единственным и далеко не самым удобным способом работы с Git. Какие Вам известны программные средства с графическим интерфейсом для работы с Git? Приведите как реализуются описанные в лабораторной работе операции Git помощью одного из таких программных средств.

GitHub Desktop, Sourcetree, GitKraken, SmartGit, плагины для сред разработки. В данной работе сделать все приведённые операции для работы с Git можно с помощью встроенных средств работы с Git в VisualStudio. Операции проводятся с помощью графического интерфейса.

Выводы: в результате выполнения данной работы были на практике исследованы основные возможности системы контроля версий Git и веб-сервиса для хостинга проектов GitHub.